University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Final Examination, January 2017

Read through the entire examination paper before you start writing so that you can answer the easy questions first. The relative importance of a question is indicated by the marks allocated to it. Please write legibly.

This paper consists of two sections:

Section 1: Advanced Software Design — compulsory
Section 2: Compilers — compulsory

You must answer all questions for both Sections 1 and 2.

Total Marks: 90	Approximate marks per question are shown in brackets.
Time: 3 hours + 10 minutes reading	The use of calculators is permitted.

Please write your answers in two separate books. Use one book for Advanced Software Design, and one for Compilers, and label them as such on the cover page.

Your attention is drawn to the instructions on your answer book on the front and the back. In particular, "**EVERY CANDIDATE MUST enter the book number and the number of each question answered (in the order in which it has been answered)**" on the table provided on the front cover of the first book. Please **do this for the front cover of the different books used for each section.** This is needed so that we can allocate marks and not miss questions. A list of sub-questions is *not* required.

Section 1. Advanced Software Design [45 Marks]

Question 1. **[10]**

a. What problem is being solved by the Pipe and Filter design pattern and how? [6]

b. The key aim of many patterns is to minimize coupling while maximizing cohesion. Explain what cohesion and coupling are. [4]

Question 2. **[12]**

a. How does test-driven development work and what does it achieve? [6]

b. How do iterative processes accommodate risk better than the waterfall approach? [6]

Question 3. **[10]**

Rational Unified Process suggests a "4+1" view of architectures. List and briefly describe all elements of this view.

Question 4. **[13]**

a. Understanding and fostering culture is an important component of sustaining open source projects. Name some techniques and principles that should be considered in open source projects. [4]

b. Define the following terms related to version control: commit, update, repository, checkout, working copy, diff, merge, conflict, lock. [9]

Section 2. Compilers [45]

Question 1. **[18]**

a. Explain the fundamental difference between a programming language interpreter and a compiler?

[2]

b. An existing compiler for a programming language which produces code on an Intel architecture is being adapted for a new processor architecture. Which phases of the compiler will have to be rewritten? [3]

c. For the alphabet {a,b}, find a regular expression that matches all strings of odd length, starting with a. [2]

d. Given the following grammar, do a leftmost derivation the string "xxy", showing every step. [2]

$S \rightarrow AB$

$A \rightarrow Ax$

$A \rightarrow \varepsilon$

$B \rightarrow yB$

$B \rightarrow y$

e. Specify what regular expression describes the language of the grammar above? [2]

Consider the following fragments of Java code which each contain an error. Indicate what the error is and in which phase of compilation this error is identified — Lexical Analysis, Parsing(Syntactic Analysis) or Semantic Analysis.

f. [2]

```
int total = 0;
for(int i = 0; i <= 10; i++) {
        total += i;
}
System.out.println("total from 1 to " + i + " is " + total);
```

g. [2]

```
String name = "Thandi";
String email@work = "thandi@cs.uct.ac.za";
System.out.println(name + " has the work email address " + email@work);
```

h. [2]

```
int i = 10
while (i > 0) {
        System.out.println(i++);
}
else {
        System.out.println(i);
}
```

i. Indicate whether the following statement is True or False: "Bottom Up Parsing uses leftmost derivation." [1]

Question 2. **[6]**

a. Which single data structure keeps a list of all current variable bindings? [1]

b. Which data structure is particularly useful during recursion for storing local variables and parameters for each subprogram/function call? [1]

Consider the following Java program:

```
class ScopingQuestion {

  static int a = 2;
  static int b = 7;
  static int c = 13;

  static void doIt() {
    int c = 1;
    System.out.println(a+b);
    System.out.println(b+c);
  }

  static void doSomething() {
    int b = 4;
    int c = 11;
    doIt();
  }

  public static void main(String[] args) {
    int a = 3;
    int b = 5;
    doSomething();
  }
}
```

Indicate which values are displayed if the following types of scoping are used:

c. Static scoping. [2]

d. Dynamic scoping. [2]

Question 3. **[7]**

a. Consider the following IR statements. Determine the original expression which generated this code. Note that offset_a, offset_b and offset_c are the offsets from the frame pointer fp. [3]

```
MOVE(
   MEM(+(TEMP fp, CONST offset_a)),
   *(
      CONST 10,
      -(
         MEM(+(TEMP fp, CONST offset_b)),
         +(
                  MEM(+(TEMP fp, CONST offset_c)),
                     CONST 8
               )
      )
   )
)
```

b. Explain the IR Code Optimisation techniques inlining and loop unrolling. [2]

c. What other optimisation technique could be used to make the optimised code more efficient? [1]

d. Specify one disadvantage of the using an Intermediate Representation in the compilation process. [1]

Question 4. **[14]**

a. Indicate whether the following statement is True or False: “The process of code block reorganisation attempts to maintain the jumps within the IR code.” [1]

b. Explain the process of Instruction Selection. [2]

c. Name one of the algorithms used to do Instruction Selection. [1]

d. During Liveness Analysis, what does a node represent in a Control Flow Graph? [1]

e. Use liveness analysis to calculate the live-in and live-out sets for each of the statements in a program below. Show the succ, use, def, in and out sets. Hint: The relevant formulae can be stated as follows: [8]

```
out[n] = union of in[s] for all successors s of n
in[n] = union of use[n] and (out[n] – def[n])
```

```
1.  if (i>j)
2.    then i += 1;
3.    else j += 1;
4.  print(i+j);
```

f. How are pre-coloured nodes handled when drawing the interference graph during Register Allocation? [1]